AULA INVERSA

Tópicos Avançados de Compiladores

Resoluçãocomentadade5questõeselaboradas no padrão ENADE

Fluxo em camadas • Cinco problemas práticos • Distratores e erros conceituais



Integrantes

1.Diego Lemos Passani Gomes 2.Fernanda Brandão Pizzinatto 3. Miguel Fernandes Bellini 4. Nicolly de Paula Santos

Fluxo de compilação

01
Código-fonte
caracteres

›

02
Análise léxica
tokens

›

03
Análise sintática
estrutura / AST

›

04
Análise semântica
tipos, escopo e símbolos

›

05
IR e otimizações
representação intermediária

›

06
Geração de código
instruções da arquitetura-alvo

Front-end: análise do código
Caracteres, estrutura, tipos e escopo

Back-end: IR e recursos da arquitetura
Seleção de instruções e alocação de registradores

Análise léxica

Lexer: caracteres → tokens

Lexema

Texto concreto do código-fonte

```
contador1     if     >=
```

Token

Categoria atribuída ao lexema

```
IDENTIFICADOR   KW_IF   OPERADOR
```

Conflito de padrões

`letra (letra | dígito)*`

if e while também atendem à regra geral

Sequência alfanumérica → tabela de keywords → token específico

QUESTÃO 1 – ANÁLISE LÉXICA

Uma equipe de desenvolvimento está implementando o analisador léxico de uma nova linguagem de programação. A especificação da linguagem determina que os identificadores sejam reconhecidos pela seguinte regra:

```
IDENTIFICADOR → letra (letra | dígito)*
```

Durante os testes, o analisador reconheceu corretamente nomes como valor, resultado e contador1. Entretanto, também classificou os lexemas if, while e return como identificadores, embora eles estejam definidos como palavras reservadas da linguagem.

Para facilitar a manutenção do compilador, a equipe mantém as palavras reservadas em uma tabela própria, permitindo que novas palavras sejam adicionadas sem modificar a regra geral de formação dos identificadores.

A equipe precisa identificar a causa da classificação incorreta e definir uma estratégia de reconhecimento que diferencie os identificadores comuns das palavras reservadas.

DADOS DE SUPORTE

Tabela 1 – Palavras reservadas definidas pela linguagem

Lexema	Token definido
if	KW_IF
while	KW_WHILE
return	KW_RETURN

Regra geral de reconhecimento de identificadores:

```
IDENTIFICADOR → letra (letra | dígito)*
```

Durante um teste, foi processado o trecho:

```
if (contador1 >= 10)
    return resultado;
```

Tabela 2 – Saída produzida pelo analisador léxico

Lexema	Token produzido
if	IDENTIFICADOR
(	ABRE_PARENTESE
contador1	IDENTIFICADOR
>=	OPERADOR_RELACIONAL
10	NUMERO
)	FECHA_PARENTESE
return	IDENTIFICADOR
resultado	IDENTIFICADOR
;	PONTO_E_VIRGULA

Fonte: elaboração própria, adaptada conceitualmente de LLVM Project, “My First Language Frontend with LLVM Tutorial”, capítulo 5 – Lexer Extensions for If/Then/Else. llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl05.html

Considerando a regra de formação de identificadores, a tabela de palavras reservadas e a saída produzida pelo analisador léxico, assinale a alternativa que identifica corretamente a causa da classificação inadequada de if e return e apresenta uma estratégia adequada para corrigir o reconhecimento desses lexemas.

A) O analisador deve manter if e return como identificadores durante a etapa léxica e permitir que o analisador sintático determine posteriormente se esses lexemas exercem funções reservadas conforme sua posição no programa.

B) O padrão de identificadores deve ser reformulado para impedir que qualquer sequência correspondente a uma palavra reservada seja reconhecida inicialmente, eliminando a necessidade de consultar a tabela mantida pela linguagem durante a análise léxica.

C) O analisador deve reconhecer inicialmente a sequência compatível com identificadores e, em seguida, consultar a tabela de palavras reservadas, atribuindo prioridade ao token específico sempre que o lexema estiver registrado nessa tabela.

D) O analisador deve conservar a classificação de identificador e registrar na tabela de símbolos que determinados nomes possuem uso especial, permitindo que a análise semântica determine posteriormente sua função na linguagem.

Reconhecer o padrão e depois consultar a tabela

C

RESPOSTA

•O lexer reconhece a sequência compatível com IDENTIFICADOR.
Depois consulta a tabela de palavras reservadas.
Se houver correspondência, emite o token específico (ex.: KW_IF).

POR QUE AS OUTRAS NÃO?

- A transfere uma decisão lexical para o parser.

B abandona a tabela e contraria o requisito de manutenção.
D mantém a classificação lexical incorreta e empurra a decisão para a semântica.

Análise sintática

Parser: sequência de tokens conforme a gramática

Vocabulário

Terminal

Símbolo daentrada: ID, NUMERO,

vírgula, parênteses

Não terminal

Categoria:<chamada>, <lista>

Produção

Regra de formação

Somente um argumento

```
<chamada> → ID '(' <argumento> ')' ';'
```

Lista recursiva

```
<lista> → <argumento>
        | <lista> ',' <argumento>
```

Um ou mais argumentos separados por vírgulas

QUESTÃO 2 – ANÁLISE SINTÁTICA

Uma equipe de desenvolvimento está implementando o analisador sintático (parser) de uma nova linguagem destinada à automação de processos. A especificação da linguagem permite chamadas de funções com um ou mais argumentos, separados por vírgulas.

Durante os testes, o analisador léxico reconheceu corretamente todos os elementos do código. Entretanto, o parser rejeitou uma chamada de função ao encontrar a primeira vírgula entre os argumentos. A equipe precisa verificar se a falha está na sequência de tokens produzida ou nas regras da gramática e definir uma correção que não limite a quantidade de argumentos permitidos.

DADOS DE SUPORTE

Trecho de código analisado

```
registrarVenda(cliente, total, 10);
```

Gramática utilizada pelo parser

```
<chamada>   → IDENTIFICADOR '(' <argumento> ')' ';'
<argumento> → IDENTIFICADOR | NUMERO
```

Sequência de tokens produzida pelo analisador léxico

Lexema	Token produzido
registrarVenda	IDENTIFICADOR
(	ABRE_PARENTESE
cliente	IDENTIFICADOR
,	VIRGULA
total	IDENTIFICADOR
,	VIRGULA
10	NUMERO
)	FECHA_PARENTESE
;	PONTO_E_VIRGULA

Fonte: elaboração própria, adaptada conceitualmente de GNU Project, “Bison 3.8.1 Manual”, seções 3.3.1 – Syntax of Grammar Rules e 3.3.3 – Recursive Rules. https://www.gnu.org/software/bison/manual/html_node/Recursion.html

Considerando o trecho de código, a sequência de tokens e as produções da gramática, analise a causa do erro sintático e assinale a alternativa que identifica corretamente a limitação da gramática e apresenta uma alteração capaz de reconhecer chamadas com um ou mais argumentos separados por vírgulas.

A) A sequência de tokens está inadequada porque as vírgulas deveriam ser descartadas pelo analisador léxico. Mantendo a produção de chamada inalterada, o parser poderia processar os argumentos consecutivos como uma única construção sintática.

B) O erro ocorre porque a produção de chamada admite apenas um argumento. Deve-se introduzir uma lista recursiva, como <lista> → <argumento> | <lista> ',' <argumento>, e usar <lista> no interior dos parênteses da produção <chamada>.

C) O erro ocorre porque a produção atual não prevê três argumentos. Deve-se substituir <chamada> por uma produção contendo exatamente três <argumento> separados por vírgulas, permitindo que o trecho apresentado seja reconhecido pelo parser.

D) A gramática da chamada pode permanecer inalterada, desde que o parser aceite os tokens existentes entre parênteses e transfira à análise semântica a tarefa de separar os argumentos e verificar sua quantidade conforme a função utilizada.

A gramática precisa representar uma lista

B

RESPOSTA

•<lista> → <argumento> | <lista> "," <argumento>
Caso-base: 1 argumento · Recursão: amplia a lista.
<chamada> passa a usar <lista> entre parênteses.

POR QUE AS OUTRAS NÃO?

- A remove uma vírgula que é separador sintático necessário.

C resolve apenas o caso com três argumentos.
D a análise semântica não substitui uma estrutura ausente na gramática.

Análise semântica

Sintaxe correta não garante significado válido

Verificações
Compatibilidade de tipos
Declaração de variáveis
Escopo

Tabela de símbolos
Nomes, tipos e permissões

Exemplo
```
quantidade = produto - 5;
```

produto: texto Subtração: inteiros

Conversão explícita
Na questão: sem conversão implícita entre texto e inteiro

```
paraInteiro(produto) - 5
```

QUESTÃO 3 – ANÁLISE SEMÂNTICA

Uma equipe de desenvolvimento está testando o compilador de uma linguagem estaticamente tipada utilizada em um sistema de automação comercial. O trecho abaixo foi aceito pelas etapas de análise léxica e sintática, mas a análise semântica identificou uma inconsistência na expressão atribuída à variável quantidade.

A linguagem não realiza conversão implícita entre texto e inteiro em operações aritméticas. Quando um valor textual representa um número e precisa participar de uma operação numérica, a especificação exige o uso explícito da função de conversão paraInteiro(texto). A equipe precisa diagnosticar a inconsistência e indicar uma correção compatível com essas regras.

DADOS DE SUPORTE

Trecho de código analisado

```
inteiro quantidade = 10;
texto produto = "25";
quantidade = produto - 5;
```

Tabela de símbolos

Identificador	Tipo declarado
quantidade	inteiro
produto	texto

Regras semânticas relevantes da linguagem

1. O operador de subtração (-) exige operandos do tipo inteiro.
2. Uma variável inteira só pode receber uma expressão cujo resultado seja inteiro.
3. A conversão de texto numérico para inteiro deve ser explícita por meio de paraInteiro(texto).

Fonte: elaboração própria, adaptada conceitualmente de Oracle, Java Language Specification, Java SE 26, seções 5.2 (Assignment Contexts) e 15.18 (Additive Operators). https://docs.oracle.com/javase/specs/jls/se26/html/

Considerando o trecho de código, a tabela de símbolos e as regras semânticas apresentadas, analise a inconsistência e assinale a alternativa que identifica corretamente o erro e propõe uma correção que torne o trecho semanticamente válido.

A) A inconsistência é lexical, pois o literal "25" contém apenas dígitos e deveria ser classificado como número mesmo entre aspas; reconhecendo-o como NUMERO, a operação com produto poderia ser mantida sem conversão adicional.

B) A inconsistência é sintática, pois a gramática não deveria permitir a subtração entre um identificador textual e um número; acrescentando uma produção específica para texto - inteiro, a expressão passaria a ser semanticamente válida.

C) A expressão pode ser mantida, pois o gerador de código deve detectar que o conteúdo de produto é numérico e realizar automaticamente a conversão para inteiro antes da subtração, preservando o tipo da variável de destino.

D) A análise semântica deve rejeitar produto - 5 porque produto é texto e a subtração exige inteiros; para atender às regras, a expressão pode ser corrigida para quantidade = paraInteiro(produto) - 5, tornando os tipos compatíveis.

QUESTÃO 3 · CORREÇÃO

Sintaxe válida não garante tipos válidos

D

RESPOSTA

- produto é texto; 5 é inteiro.

A subtração exige dois operandos inteiros.
Correção: quantidade = paraInteiro(produto) - 5;

POR QUE AS OUTRAS NÃO?

- A "25" entre aspas continua sendo texto.

B mudar a gramática não torna tipos incompatíveis válidos.
C conversão automática no gerador contraria a especificação.

Geração de código e registradores

Back-end: IR → instruções da máquina

Temporários virtuais

`IR: t1, t2, t3, …`

Semcorrespondênciadiretacom registradores físicos

Registradores físicos

Conjunto limitado pelo processador

Na questão: apenas R1 e R2

Liveness

Valor ainda necessário
em uma operação futura

Spill

Salvar em memória e reutilizar o registrador

```
STORE TEMP, R1
LOAD R2, TEMP
```

QUESTÃO 4 – GERAÇÃO DE CÓDIGO

Uma equipe está implementando o back-end de um compilador para um processador simples que dispõe de apenas dois registradores físicos de uso geral, R1 e R2. A representação intermediária do programa utiliza temporários virtuais sem essa limitação, mas a etapa de geração de código precisa produzir uma sequência válida para a arquitetura de destino.

Durante a tradução, os dois resultados intermediários t1 e t2 precisam ser preservados até a multiplicação final. Como não há registradores físicos suficientes para manter simultaneamente todos os valores necessários, a arquitetura permite armazenar temporariamente um valor na posição de memória TEMP.

DADOS DE SUPORTE

Representação intermediária

```
t1 = a + b
t2 = c + d
t3 = t1 * t2
resultado = t3
```

Instruções disponíveis na arquitetura

Instrução	Efeito
LOAD Rn, X	Rn <- valor de X
STORE X, Rn	X <- valor de Rn
ADD Rd, Rs	Rd <- Rd + Rs
MUL Rd, Rs	Rd <- Rd * Rs

Restrições: somente R1 e R2 existem como registradores físicos; qualquer uso de R3 é inválido. A posição TEMP pode ser utilizada para spill temporário em memória.

Fonte: elaboração própria, adaptada conceitualmente de LLVM Project, “The LLVM Target-Independent Code Generator”, seções Instruction Selection e Register Allocation. https://llvm.org/docs/CodeGenerator.html

Considerando a representação intermediária, o conjunto de instruções e as restrições da arquitetura, avalie as estratégias abaixo e assinale a sequência que preserva corretamente a semântica do programa e utiliza apenas os recursos disponíveis no processador.

A) LOAD R1,a; LOAD R2,b; ADD R1,R2; LOAD R3,c; LOAD R2,d; ADD R3,R2; MUL R1,R3; STORE resultado,R1. A sequência preserva t1 e t2 em registradores e evita acesso temporário à memória.

B) LOAD R1,a; LOAD R2,b; ADD R1,R2; STORE TEMP,R1; LOAD R1,c; LOAD R2,d; ADD R1,R2; LOAD R2,TEMP; MUL R2,R1; STORE resultado,R2. A sequência usa TEMP para preservar t1 e respeita os dois registradores disponíveis.

C) LOAD R1,a; LOAD R2,b; ADD R1,R2; STORE TEMP,R1; LOAD R2,TEMP; LOAD R1,c; LOAD R2,d; ADD R1,R2; MUL R2,R1; STORE resultado,R2. A sequência realiza spill, mas reutiliza R2 antes da multiplicação final.

D) LOAD R1,a; LOAD R2,b; ADD R1,R2; STORE TEMP,R1; LOAD R1,c; LOAD R2,d; ADD R1,R2; LOAD R2,TEMP; ADD R2,R1; STORE resultado,R2. A sequência respeita os registradores e combina os dois resultados intermediários ao final.

Alternativa B · simulação de R1, R2 e TEMP

B

Passo	Instrução	R1	R2	TEMP
1	`LOAD R1, a`	a	—	—
2	`LOAD R2, b`	a	b	—
3	`ADD R1, R2`	t1 = a + b	b	—
4	`STORE TEMP, R1`	t1	b	t1 (spill)
5	`LOAD R1, c`	c	b	t1
6	`LOAD R2, d`	c	d	t1
7	`ADD R1, R2`	t2 = c + d	d	t1
8	`LOAD R2, TEMP`	t2	t1	t1
9	`MUL R2, R1`	t2	t3 = t1 * t2	t1
10	`STORE resultado, R2`	t2	t3	t1

Passo 4: spill · Passo 8: reload · Passo 9: multiplicação · R3 nunca pode ser usado

Ambiguidade, precedência e associatividade

Gramática ambígua: mais de uma árvore para a mesma entrada

Precedência
Prioridade entre operadores diferentes

`* antes de +`

Associatividade
Agrupamento no mesmo nível

`a - b - c → (a - b) - c`

GNU Bison

```
%left '+' '-'
%left '*'
```

Declaração posterior: maior precedência
%left: associatividade à esquerda

QUESTÃO 5 – AMBIGUIDADE, PRECEDÊNCIA E ASSOCIATIVIDADE

Uma equipe está desenvolvendo o parser de uma linguagem para expressões aritméticas. Durante a geração do analisador sintático, a gramática abaixo produz conflitos porque uma mesma expressão pode admitir diferentes árvores sintáticas. O problema tornou-se evidente ao testar a expressão a + b * c - d.

A especificação da linguagem estabelece que multiplicação possui precedência maior que adição e subtração, enquanto adição e subtração têm a mesma precedência e são associativas à esquerda. A equipe precisa escolher a interpretação correta e uma estratégia de configuração do parser que elimine o conflito de acordo com essas regras.

DADOS DE SUPORTE

Gramática utilizada

```
expr -> expr '+' expr
      | expr '-' expr
      | expr '*' expr
      | ID
```

Precedência e associatividade definidas pela linguagem

Operadores	Precedência	Associatividade
+ -	menor	esquerda
*	maior	esquerda

Interpretações possíveis para a expressão

I. ((a + b) * c) - d
II. (a + (b * c)) - d
III. a + (b * (c - d))

Fonte: elaboração própria, adaptada conceitualmente de GNU Project, “Bison 3.8.1 Manual”, seções 5.3.2 – Specifying Operator Precedence, 5.3.3 – Specifying Precedence Only e 5.3.4 – Precedence Examples. https://www.gnu.org/software/bison/manual/

Considerando a gramática, as regras de precedência e associatividade e as interpretações apresentadas, avalie o conflito e assinale a alternativa que identifica corretamente a ambiguidade, seleciona a interpretação prevista pela linguagem e apresenta uma estratégia adequada para configurar o parser.

A) A gramática é ambígua e a interpretação I deve ser escolhida, pois a associatividade à esquerda determina a redução de todos os operadores na ordem em que aparecem; declarar %left '+' '-' '*' em um único nível resolve o conflito.

B) A gramática não é ambígua, pois a ordem dos operadores na cadeia de entrada determina uma única derivação; a interpretação II surge automaticamente e, por isso, não é necessário declarar precedência nem associatividade no gerador do parser.

C) A gramática é ambígua e a interpretação II deve prevalecer; no Bison, pode-se declarar %left '+' '-' e, em uma declaração posterior, %left '*', atribuindo maior precedência à multiplicação e associatividade à esquerda.

D) A gramática é ambígua e a interpretação III deve prevalecer, pois a multiplicação deve incorporar toda a expressão à sua direita; declarar %right '+' '-' antes de %left '*' garante essa associação e elimina os conflitos.

Alternativa C · multiplicação com precedência maior

C

CONFIGURAÇÃO

```
%left '+' '-'
%left '*'
```

Declaração posterior no Bison → maior precedência

INTERPRETAÇÃO II

```
(a + (b * c)) - d
```

* agrupa primeiro; + e - ficam no mesmo nível e associam à esquerda.

A mesmo nível para +, - e *

B ordem dos tokens não elimina a ambiguidade

D interpretação III + associatividade à direita contrariam as regras

Resumo integrado

1	**Léxica**	Lexema → token correto?	**Tabela de reservadas**
2	**Sintática**	A sequência é derivável pela gramática?	**Lista recursiva**
3	**Semântica**	Tipos, declarações e usos são compatíveis?	**Conversão explícita**
4	**Geração de código**	Temporários cabem nos recursos físicos?	**Spill / reload**
5	**Ambiguidade**	Qual árvore respeita as regras da linguagem?	**Precedência / associatividade**